

DNS Walkthrough — yuguda.flyrank.ai (future subdomain)

What a CNAME record is

A CNAME ("Canonical Name") record is a DNS alias. It doesn't point a domain at a server's IP address directly — it points one domain name at *another* domain name, and says "whatever answers for that other name, use that answer for me too." It's the DNS equivalent of a forwarding label: instead of writing your street address on every letter, you just write "same address as this other person," and whoever's sorting mail follows that label to the real address.

What value mine will hold

Once Ops provisions `yuguda.flyrank.ai`, they'll create a CNAME record:

```
yuguda.flyrank.ai.    CNAME    <my-site-name>.netlify.app.
```

`<my-site-name>.netlify.app` is my site's permanent Netlify address — the one I renamed away from the random spontaneous-kitten-style default. The CNAME means `yuguda.flyrank.ai` never has to point at a raw IP address; it just always resolves to wherever Netlify is currently serving my site, even if Netlify's own infrastructure changes underneath it.

What actually happens when someone types the address

1. **Typing it in.** Someone types `yuguda.flyrank.ai` into their browser. The browser doesn't know where that is yet — it needs an IP address.
2. **Asking a resolver.** The browser asks a DNS *resolver* (usually run by the person's ISP, or a public one like 1.1.1.1) to look up `yuguda.flyrank.ai`. The resolver is like a phone operator: it doesn't personally know every number, but it knows who to call to find out.
3. **Walking the nameservers.** The resolver asks the root nameservers "who handles `.ai`?", then asks the `.ai` nameservers "who handles `flyrank.ai`?", and finally asks FlyRank's *authoritative nameserver* "what's the record for `yuguda.flyrank.ai`?" This is the actual phonebook lookup, one level at a time.
4. **Getting the record.** FlyRank's nameserver answers with the CNAME record: "that name is an alias for `<my-site-name>.netlify.app`." The resolver isn't done yet — a CNAME is a pointer, not a final answer, so it now has to look up *that* name too, the same way, until it lands on an actual IP address (Netlify's).
5. **Getting the response.** The resolver hands that IP address back to the browser. The browser connects to Netlify's server at that address, asks specifically for `yuguda.flyrank.ai` (so Netlify knows which of its many hosted sites to serve), and Netlify's own SSL certificate for that domain lets the connection happen over HTTPS — that's the padlock.
6. **Confirming the padlock.** If every step above resolved correctly and Netlify has issued a certificate for `yuguda.flyrank.ai` specifically, the browser shows the padlock with no warnings. If the CNAME hasn't propagated yet, or the custom domain hasn't been added in Netlify's settings, this is the step that fails — which is exactly what the domain-troubleshooting guide checks first.

Why I'm writing this now, not later

None of this needs to happen today — the subdomain isn't provisioned until the capstone is approved. But by then I want this to be a checklist, not a research task: add the custom domain in Netlify, confirm the CNAME value Ops gave me matches what's actually in my site settings, wait for propagation, check the padlock. Nothing about the site itself changes — a custom domain is a new label pointing at the same mailbox.